

WEEK 2: PRODUCT AND CATEGORY MANAGEMENT SYSTEM

Scenario:

An online store wants to manage information about different brands and the items they sell.

The system stores item details such as item name, price, stock quantity, and the brand each item belongs to.

1. Design product and category tables:

Category Table:

```
SQL> CREATE TABLE Category(  
2     Category_ID VARCHAR2(7),  
3     Category_Name VARCHAR2(20),  
4     Description VARCHAR2(50)  
5 );
```

Table created.

Product Table:

```
SQL> CREATE TABLE Product(  
2     Product_ID VARCHAR2(7),  
3     Product_Name VARCHAR2(20),  
4     Category_ID VARCHAR2(7),  
5     Price NUMBER(9,2),  
6     Stock_Quantity Number  
7 );
```

Table created.

Create separate tables to store product details and category information.

Each product will be associated with a specific category.

2. Define Relationships and Constraints

Relationships:

One category can contain multiple products.

The relationship between the **Category** table and the **Product** table is **One-to-Many** (1 → *).

To create this relationship, we need to identify the keys that uniquely identify records and connect the two tables.

PRIMARY KEY: A column that uniquely identifies each record in a table.

In the **Category** table, Category_ID uniquely identifies each category. In the **Product** table, Product_ID uniquely identifies each product.

FOREIGN KEY: A key used to establish a connection between two related

tables. Here, Category_ID in the **Product** table acts as the **Foreign Key** and connects each product to its corresponding category.

NOT NULL: Ensures that a column must always contain a value and cannot be left empty.

For example, the **Product Name** and **Category Name** should always be provided.

CHECK: Ensures that the values entered in a column follow a specified condition.

For example, the **Product Price** should always be greater than zero, and stock quantity should not be negative.

UNIQUE: Ensures that duplicate values are not allowed in a column. For example, each **Category Name** should be different, so two categories cannot have the same name.

Defining Relationship and Creating Constraints for Product and Category:

Primary Key:

```
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);

Table altered.
```

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);

Table altered.
```

Creating Relationship using **FOREIGN KEY**:

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT FK_Product_Category
  3  FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);

Table altered.
```

Adding Constraints: NOT NULL:

```
SQL> ALTER TABLE Category
  2  MODIFY Category_Name VARCHAR(100) NOT NULL;

Table altered.
```

```
SQL> ALTER TABLE Product
  2  MODIFY Product_Name VARCHAR(150) NOT NULL;

Table altered.
```

CHECK:

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Price CHECK (Price > 0);

Table altered.
```

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);

Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);

Table altered.
```

3. Insert Values:

```
SQL> INSERT INTO Category
  2  VALUES(
  3      '&Category_ID',
  4      '&Category_Name',
  5      '&Description'
  6  );
Enter value for category_id: Cat0001
old   3:      '&Category_ID',
new   3:      'Cat0001',
Enter value for category_name: Electronics
old   4:      '&Category_Name',
new   4:      'Electronics',
Enter value for description: Electronic gadgets and devices
old   5:      '&Description'
new   5:      'Electronic gadgets and devices'

1 row created.

SQL> /
Enter value for category_id: Cat0002
old   3:      '&Category_ID',
new   3:      'Cat0002',
Enter value for category_name: Clothing
old   4:      '&Category_Name',
new   4:      'Clothing',
Enter value for description: Men,Women and Kids Apparels
old   5:      '&Description'
new   5:      'Men,Women and Kids Apparels'

1 row created.
```

Product Table:

```
SQL> INSERT INTO Product
  2  VALUES(
  3      '&ProductID',
  4      '&Product_Name',
  5      '&CategoryID',
  6      &Price,
  7      &Stock_Quantity
  8  );
Enter value for productid: P1001
old 3:  '&ProductID',
new 3:  'P1001',
Enter value for product_name: Laptops
old 4:  '&Product_Name',
new 4:  'Laptops',
Enter value for categoryid: Cat0001
old 5:  '&CategoryID',
new 5:  'Cat0001',
Enter value for price: 67500
old 6:  &Price,
new 6:  67500,
Enter value for stock_quantity: 2
old 7:  &Stock_Quantity
new 7:  2

1 row created.
```

Like this insert few more rows.

After Insertion Category Table:

```
SQL> SELECT * FROM Category;

CATEGOR
-----
CATEGORY_NAME
-----
DESCRIPTION
-----
Cat0001
Electronics
Electronic gadgets and devices

Cat0002
Clothing
Men,Women and Kids Apparels

CATEGOR
-----
CATEGORY_NAME
-----
DESCRIPTION
-----
Cat0003
Home Appliances
Household appliances

Cat0004
Books

CATEGOR
-----
CATEGORY_NAME
-----
DESCRIPTION
-----
All Genre of Books
```

Product Table:

```
SQL> SELECT * FROM Product;
```

PRODUCT

PRODUCT_NAME

CATEGOR	PRICE	STOCK_QUANTITY
---------	-------	----------------

P1001		
Laptops	67500	2
Cat0001		

P3002		
Washing Machine	18000	12
Cat0003		

PRODUCT

PRODUCT_NAME

CATEGOR	PRICE	STOCK_QUANTITY
---------	-------	----------------

P1002		
Smart Phones	25000	5
Cat0001		

P2001		
Shoes		

PRODUCT

PRODUCT_NAME

CATEGOR	PRICE	STOCK_QUANTITY
---------	-------	----------------

Cat0002	1400	5
---------	------	---

P2002		
Saree	499	10
Cat0002		

P3001		
-------	--	--

PRODUCT

PRODUCT_NAME

CATEGOR	PRICE	STOCK_QUANTITY
---------	-------	----------------

Television	42000	2
Cat0003		

P4001		
Dune	599	4
Cat0004		

PRODUCT

PRODUCT_NAME

CATEGOR	PRICE	STOCK_QUANTITY
---------	-------	----------------

P4002		
Thulasinadam	250	2
Cat0004		

--	--	--

--	--	--

8 rows selected.

CRUD Operations:

Insert Record:

```
SQL> INSERT INTO Product (Product_ID, Product_Name, Category_ID, Price, Stock_Quantity)
```

```
2 VALUES ('P3003', 'Washing Machine', 'Cat0003', 18000, 12);
```

```
1 row created.
```

Update Price:

```
SQL> UPDATE Product
```

```
2 SET Price = 60000
```

```
3 WHERE Product_Name = 'Laptop';
```

```
1 row updated.
```

Delete a Product:

```
SQL> DELETE FROM Product WHERE Product_ID = 'P3002';  
  
1 row deleted.
```

5. Generate Category-wise Product Reports

Write SQL queries to:

1) Display all products under each category.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price, p.Stock_Quantity  
2 FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID;
```

CATEGORY_NAME

PRODUCT_NAME

PRICE STOCK_QUANTITY

Electronics
Laptop
60000 5

Electronics
Smart Phones
25000 5

CATEGORY_NAME

PRODUCT_NAME

PRICE STOCK_QUANTITY

Clothing
Shoes
1400 5

Clothing
Saree

CATEGORY_NAME

PRODUCT_NAME

PRICE STOCK_QUANTITY

499 10

Home
Television
42000 2

Books

CATEGORY_NAME

PRODUCT_NAME

PRICE STOCK_QUANTITY

Dune
599 4

Books
Thulasinadam
250 2

CATEGORY_NAME

PRODUCT_NAME

PRICE STOCK_QUANTITY

Home
Washing Machine
18000 12

8 rows selected.

2) Count number of products in each category.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count
  2   FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID
  3   GROUP BY c.Category_Name;
```

CATEGORY_NAME	PRODUCT_COUNT
---------------	---------------

Clothing	2
----------	---

Books	2
-------	---

Electronics	2
-------------	---

Home	2
------	---

--	--

--	--

--	--

--	--

--	--

--	--

3) Find highest-priced product in each category.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price AS Highest_Price
2  FROM Product p
3  JOIN Category c ON p.Category_ID = c.Category_ID
4  WHERE p.Price = (
5      SELECT MAX(p2.Price)
6      FROM Product p2
7      WHERE p2.Category_ID = p.Category_ID
8  );
```

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
Electronics	Laptop	60000
Clothing	Shoes	1400
Home	Television	42000
Books	Dune	599

4) Display categories having more than 5 products.

```
SQL> SELECT c.Category_Name
2  FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID
3  GROUP BY c.Category_Name
4  HAVING COUNT(p.Product_ID) > 5;
```

CATEGORY_NAME
Electronics

5) Find average product price category-wise.

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price
  2  FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID
  3  GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
```

```
-----
  AVG_PRICE
```

```
-----
Clothing
```

```
949.5
```

```
Books
```

```
424.5
```

```
Electronics
```

```
30000
```

```
CATEGORY_NAME
```

```
-----
  AVG_PRICE
```

```
-----
Home
```

```
30000
```